

GIT

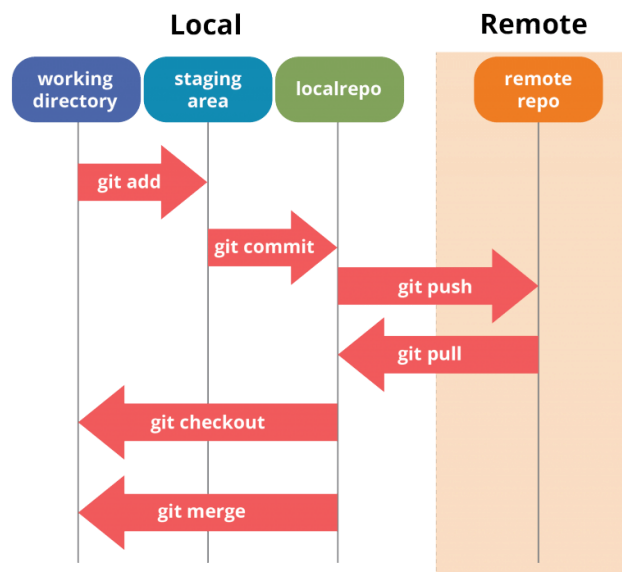
Git is a free and open-source distributed version control system (VCS) used to track changes in source code during software development. It helps developers collaborate, manage different versions of code, and roll back to previous states if needed.

- **Collaboration:** Multiple developers can work together and merge changes easily.
- **History Tracking:** Revert to previous versions whenever needed.
- **Branching & Merging:** Develop features separately and merge them safely.
- **Distributed Development:** Each developer has a full copy of the repo.
- **Incremental Changes:** Stores snapshots of the entire tree and optimizes storage internally using compression and delta encoding.

Git Architecture at a High Level

Git's architecture revolves around three core components:

1. **Working Directory** — where you edit files.
2. **Staging Area (Index)** — a snapshot of changes you plan to commit.
3. **Repository (.git folder)** — where all commits and history are stored.



Core Concepts of Git

Before using Git, it is important to understand some of its core concepts. These concepts will help you get started and make it easier to work with Git in real-world scenarios.

1. Repositories

A repository (or repo) is a storage space where your project files and their history are kept. There are two types of repositories in Git:

- **Local Repository:** A copy of the project on your local machine.
- **Remote Repository:** A version of the project hosted on a server, often on platforms like GitHub, GitLab, or Bitbucket.

2. Commits

A commit is a snapshot of your project at a specific point in time. Each commit has a unique identifier (hash) and includes a message describing the changes made. Commits allow you to track and review the history of your project.

3. Branches

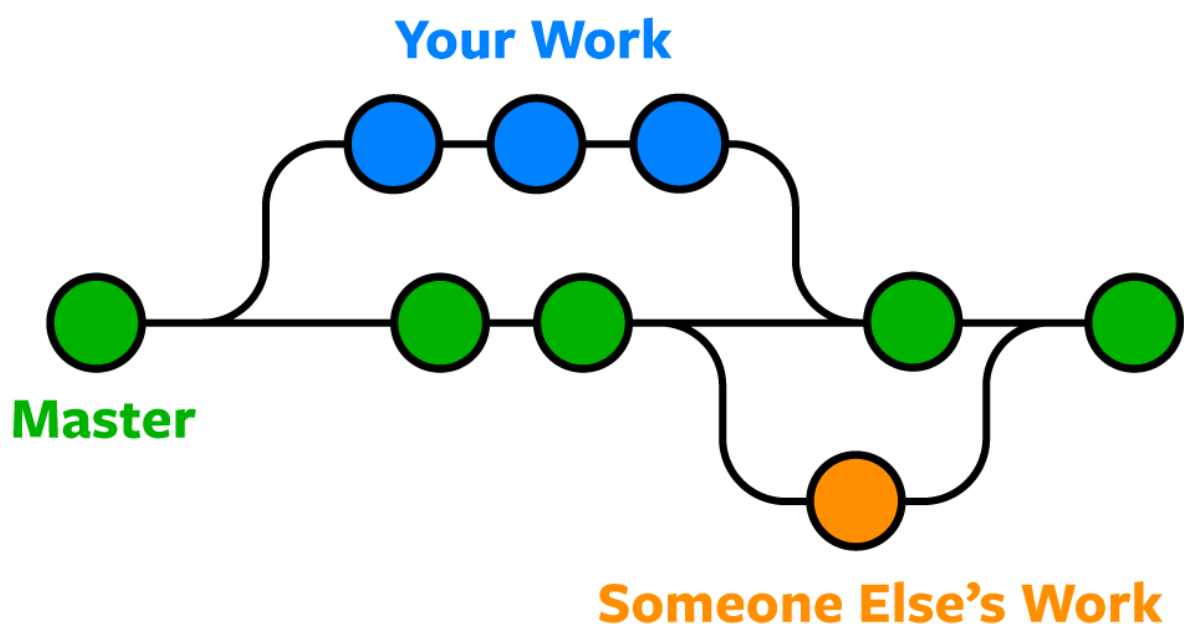
Branches allow developers to work on separate tasks without affecting the main codebase. Common branch types include:

- **Main (or Master) Branch:** The stable version of the project, usually production-ready.
- **Feature Branch:** Used for developing new features or bug fixes.

Branches

Branches allow developers to work on separate tasks without affecting the main codebase. Common branch types include:

- **Main (or Master) Branch:** The stable version of the project, usually production-ready.
- **Feature Branch:** Used for developing new features or bug fixes.



4. Merging

Merging is the process of integrating changes from one branch into another. It allows you to combine the work done in different branches and resolve any conflicts that arise.

5. Pull and Push

- **Pull:** Fetches updates from the remote repository and integrates them into your local repository.
- **Push:** Sends your local changes to the remote repository, making them available to others.

Git Commands

Git commands are important for navigating and controlling your project repository. These commands help you manage files, track changes, and collaborate with others.

Command	Description
git status	Shows the current status of the repository — staged, unstaged, and untracked files.
git add <file-name>	Stages a specific file for commit. Use git add . to stage all changes.
git commit -m "message"	Commits the staged changes with a descriptive commit message.
git commit -am "message"	Stage and commit files
git commit --amend -m "message"	Override last commit without Creating new commit
git branch <branch-name>	Creates a new branch with the given name.

Command	Description
git checkout <branch-name>	Switches to the specified branch.
git merge <branch-name>	Merges changes from the given branch into the current branch.
git push origin <branch-name>	Pushes the local branch changes to the remote repository.
git pull origin <branch-name>	Fetches and merges changes from the remote repository into the local branch.
git log	Displays the commit history for the current branch.
git log --oneline	View Short logs
git restore filename	Restore file from the last commit
git restore --source=commitId filename	Restore from Particular commit
git reset --soft <commit>	Moves the HEAD pointer to the specified commit, but keeps all changes staged.
git reset --hard <commit>	Moves HEAD and discards all changes in both the staging area and the working directory.
git reset HEAD~1	Undoes the most recent commit, keeping changes unstaged in the working directory.